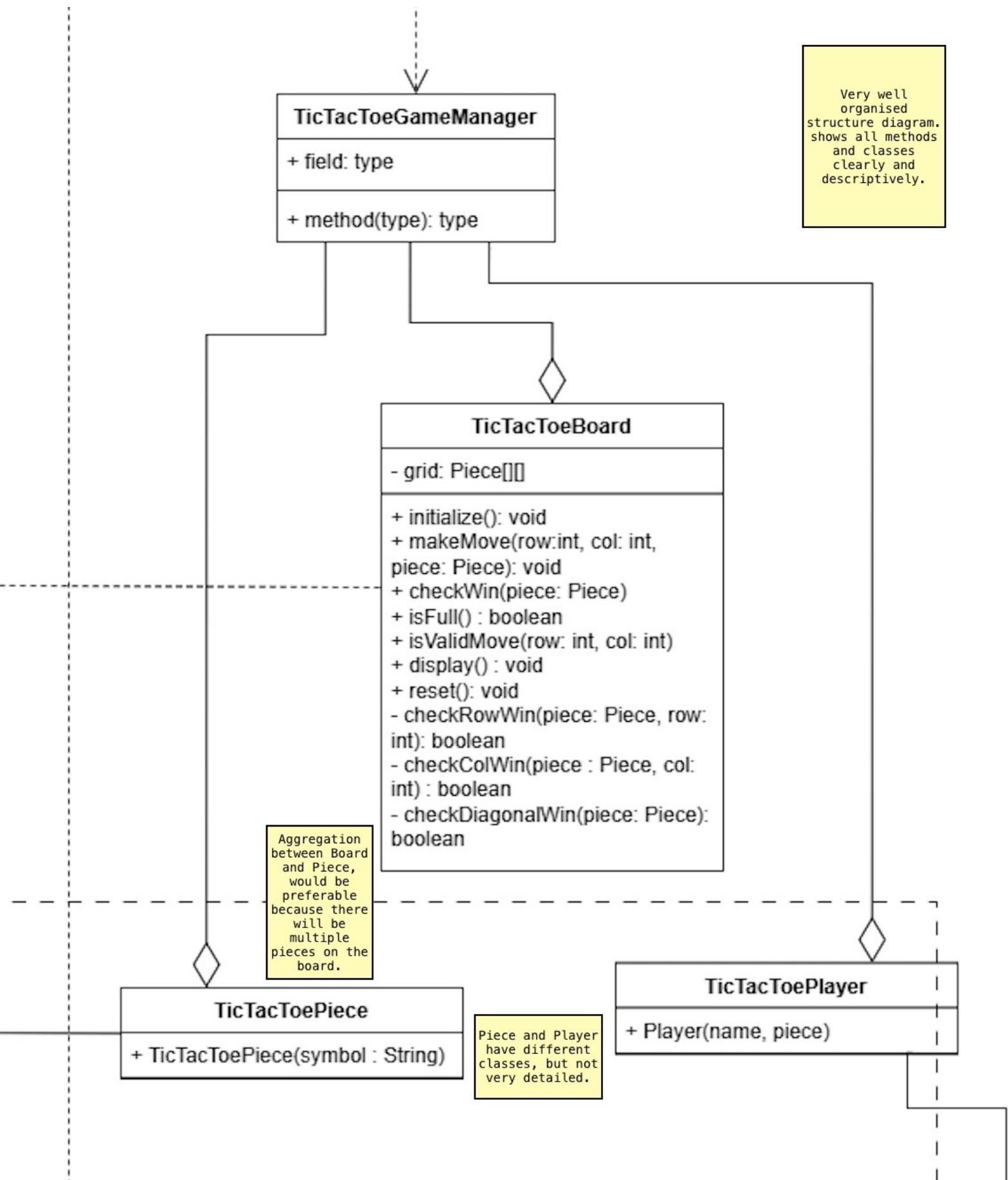
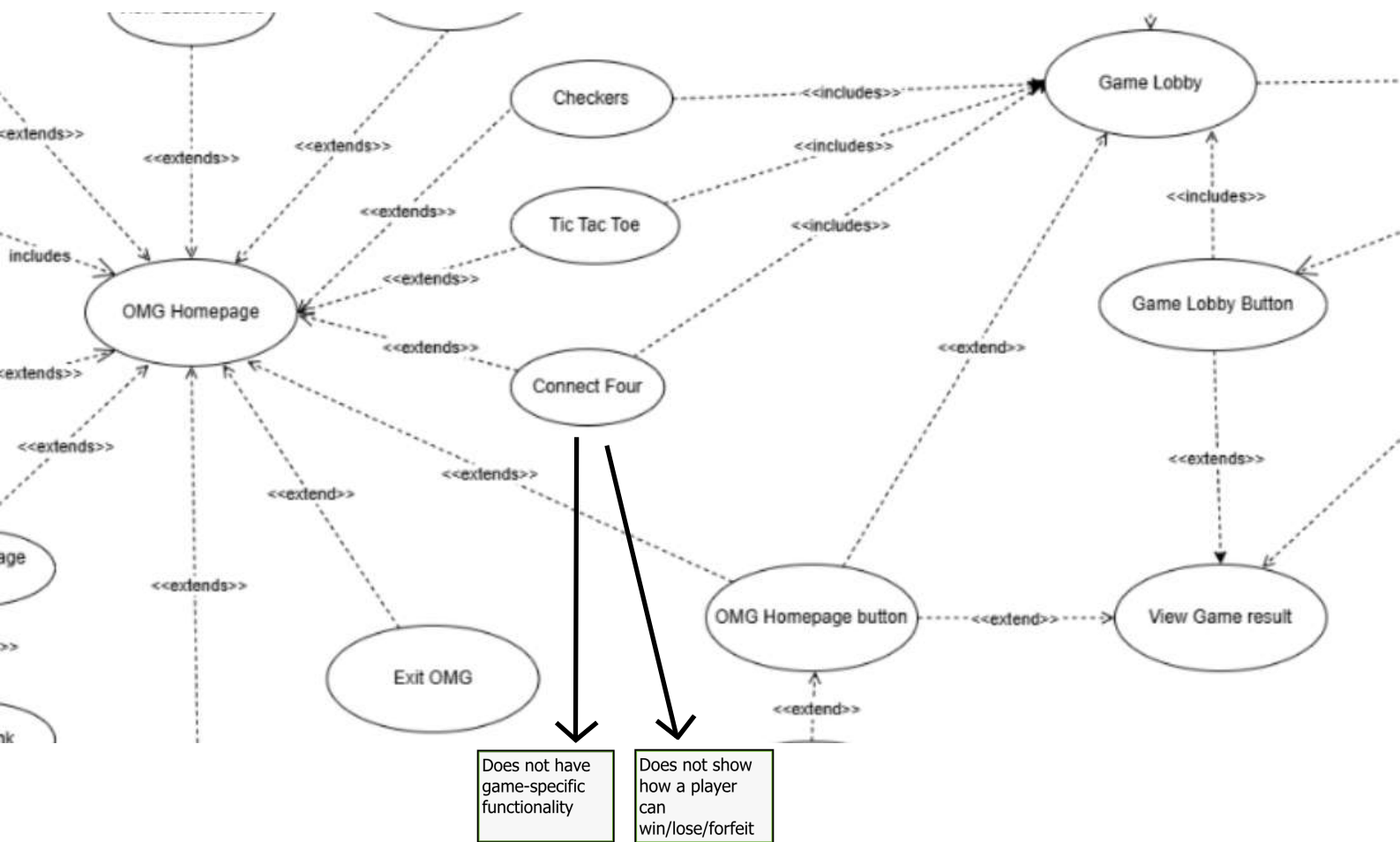






21. Use Case: Play Online

Iteration: 1

Primary Actor: User

Goal: Enable users to play against real opponents over the internet.

Preconditions: The user must be logged in. A stable internet connection is required.

Trigger: The user selects "Play Online" from the Game HomePage.

Scenario:

1. The user selects "Play Online" mode.
2. The system prompts the user to either queue for a random match or challenge a specific player.
3. The system finds an opponent or connects to the challenged player.
4. Both players enter the game lobby.
5. The system starts the game.
6. Players take turns making moves.
7. In-game features like chat, draw offers, and surrender options are available.
8. The game continues until a win/loss/draw condition is met.
9. The system updates the leaderboard and displays the result.
10. The user can choose to play again or exit.

Exceptions:

1. If the opponent disconnects, the game may be forfeited or paused.
2. If the internet connection is lost, the user may be disconnected.
3. If no opponent is found within a set time, the user can choose to expand the search or cancel.

Consider adding a reconnect grace period where a player can resume a game if their connection is restored within a short window, rather than being immediately disconnected.

Priority: High

When available: Initial release

Frequency of use: High

Channel to actor: GUI / (Play Online Button)

Secondary actors: Opponent player, Game Server

Channels to secondary actors: Internet, game matchmaking system

Open issues:

1. Should there be a ranking system for matchmaking?
2. Should users be allowed to reconnect if disconnected?

The answer to this one is yes, considering it is outlined in the scope of the project.

26. Use Case: Offer Draw Button (During Game)

Iteration: 1

Primary Actor: Player

Goal: Allow a player to offer a draw to their opponent during an ongoing game.

Preconditions:

- The player must be in an active game session.
- The game must allow draw offers (not in a checkmate or win/loss state).
- Both players must be online.

Trigger:

- The player clicks the "Offer Draw" button in the game interface.

Scenario:

1. The player clicks the "Offer Draw" button.
2. The system sends a draw offer request to the opponent.
3. The opponent receives a notification of the draw offer.
4. The opponent can either accept or decline the offer.

5. If the opponent accepts:
 - The game ends in a draw.
 - The system updates the match result and player statistics accordingly.
6. If the opponent declines:
 - The game continues as usual.
 - A notification is sent to the offering player informing them of the decline.

Consider adding a confirmation message to both players before officially ending the match, to prevent accidental acceptances due to misclicks.

Exceptions:

What happens if the opponent experiences network lag or temporary disconnection? Should the draw request remain pending for a certain duration or auto-cancel?

1. If the opponent disconnects before responding, the draw offer is automatically canceled.
2. If the game enters a checkmate or timeout state before the opponent responds, the draw offer is invalidated.
3. If the player attempts to offer a draw multiple times within a short time frame, the system may impose a cooldown period.

Priority:

- High (Ensures a fair and functional draw offering system).

When Available:

- During an ongoing match (not in a final state).

Frequency of Use:

- Occasional, dependent on player behavior.

Channel to Actor:

- Graphical User Interface (GUI) - Draw Offer Button.

Secondary Actors:

- **Opponent:** Receives the draw offer notification and responds.
- **Game System:** Processes the draw request, updates match results, and enforces game rules.

Channels to Secondary Actors:

- Opponent receives a system notification regarding the draw offer.
- Game system updates match results based on the opponent's response.

Open Issues:

1. Determine if repeated draw offers should be limited to prevent spamming.
2. Define whether a draw offer history should be recorded in the match logs.

Should draw offers be logged in the game history for review, in case of disputes or reports of unfair behavior?

30. Use Case: Handle Player Turn

Iteration: 1

Primary Actor: Player, System

Goal: The system manages turn-based gameplay logic for local multiplayer.

Preconditions:

- A game session is active.

- It is the player's turn.

Trigger:

- The player makes a move.

Scenario:

1. The system identifies the current player.
2. The player selects a move.
3. The system validates the move.
4. If valid, the system updates the game state.
5. The system checks if the game has ended.
6. If the game is not over, the system passes the turn to the next player.

Exceptions:

- If the move is invalid, an error message is displayed.
- If the game session is unexpectedly interrupted, the system prompts the player to restart or exit.

What happens if there is a network delay? Should the system implement a timeout for turns, or attempt to re-sync the game state?

Priority: High

When available: Iteration 1

Frequency of use: Every turn in a game session.

Channel to actor: GUI (Game Board)

Secondary actors: None

Channels to secondary actors: N/A

If a player disconnects mid-turn, should they be given a chance to reconnect before the turn is forfeited? How long should the system wait?

Open issues: Does the system need to allow players to undo their last move?

34: Use Case: End Game Session

Iteration: 1

Primary Actor: Player, System

Goal: The player ends the current game session.

Preconditions:

- A game session is active.

Trigger:

- The player selects the "End Game" option.

Scenario:

1. The player selects "End Game" from the menu.
2. The system confirms the action.
3. The system stops tracking the game session.
4. The system clears temporary game data.
5. The system returns to the main menu.

For online sessions, should the system also close network connections, log the player's disconnection, and update the server state?

Exceptions:

- If the player cancels the action, the game session remains active.
- If an error occurs while closing the session, the system prompts a restart.

If an opponent disconnects before the session officially ends, should the system wait for a reconnection window or automatically assign a win/loss outcome?

Priority: Low

When available: Iteration 1

Frequency of use: Once per game session.

Channel to actor: GUI (Game Menu)

Secondary actors: None

Channels to secondary actors: N/A

Open issues: Should the system save game results before closing a session?

40. Use case: Chat System

Iteration: 1

Primary Actor: User

Goal: Allows the user to chat with their opponent while playing a game

Preconditions:

- The user needs to be logged in to their account
- The user needs to be running any game

Trigger: The user uses the chat box to talk to their opponent

Scenario:

1. The user logs into the platform
2. The system displays the game selection panel
3. The user views the available list of games
4. The user selects the game they want to play (e.g. Checkers)
5. While playing the game, the user goes to the chat box to send a message
6. The user types up a message to send
7. The system sends the message to the opponent

Should there be a delivery status indicator (e.g., 'Sent', 'Delivered', 'Read') to confirm whether the message successfully reached the opponent?

Exceptions:

1. If the chat box fails to load, an error message is displayed
2. If the selected game is unavailable, an error message is displayed

Priority: Medium

When available: Within 4 sprints

Frequency of use: Sometimes used (every time the user wants to send a message)

Channel to actor: GUI (game panel UI)

Secondary actors: N/A

Channels to secondary actors: N/A

Open issues:

Should there be a rate limit on messages to prevent spam? For example, restricting users from sending multiple messages within a few seconds.

1. How should moderation be implemented for messages sent

43. Use Case: Forfeit Confirmation

Iteration: 1

Primary Actor: User

Goal:

- Inform the user of the consequences of leaving the game.
- Maintain game integrity by recording a loss if the player forfeits.

Should the system distinguish between a voluntary forfeit and a forced disconnection due to network issues? If so, how should this impact ranking and matchmaking?

Preconditions:

- The user must be logged in.
- The user must be in an active name.
- The Game Homepage or OMG Homepage must have been selected.

Trigger:

- The user clicks the "Game Homepage" button, then selects "Forfeit" to confirm navigation.

Scenario:

1. The user clicks the "Game Homepage" or "OMG Homepage" button while in an active game.
2. The system displays a warning popup with the following message:
WARNING
Are you sure you want to forfeit?
Leaving now will count as a loss, and your opponent will win by default.
3. The user chooses one of the options:
 - Forfeit. The system records a loss for the player if applicable and redirects them to the Game Homepage.
 - Cancel. The user remains in the game.
4. If the user forfeits, the respective homepage is displayed.

Exceptions: If a player forfeits mid-game, should their opponent gain additional points or benefits based on the game's progression (e.g., score, remaining time, or dominance in play)?

1. If the Homepage is down or inaccessible, the system displays an error message and prevents the user from exiting.
2. If the user cancels the warning prompt, they remain in the game.
3. If the system crashes mid-transition, the match might not be properly recorded as a loss, creating an unfair advantage.

Priority: High

When available: Sprint 1

Frequency of use: Frequent whenever the user wants to navigate back to a Homepage.

Channel to actor: GUI button on the interface.

Secondary Actors:

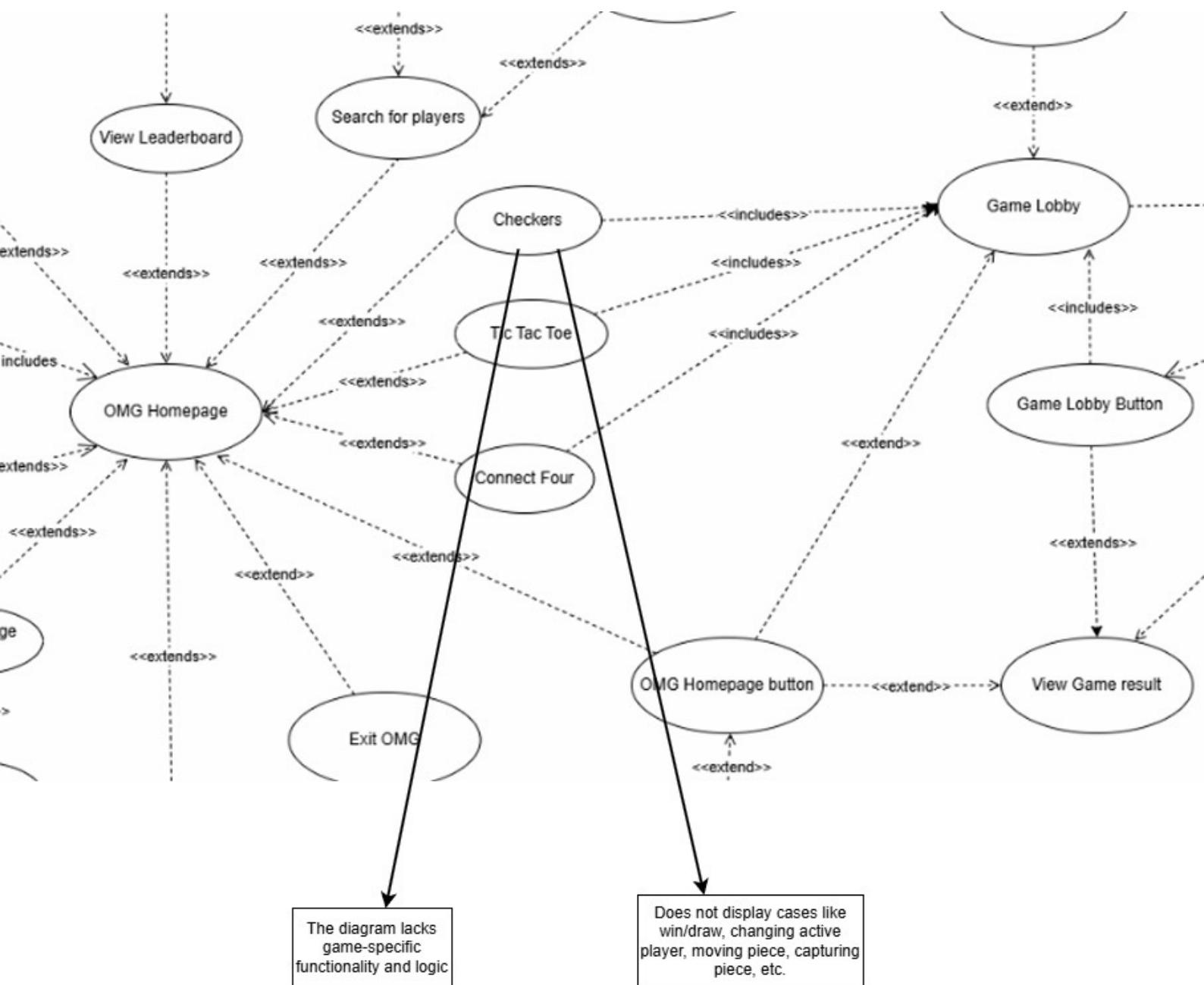
- System
 - Registers a forfeit loss if applicable.
 - Updates rankings when a forfeit occurs.
 - Notifies the opponent that their opponent forfeited.
 - Removes the player from matchmaking if they were in an online queue.

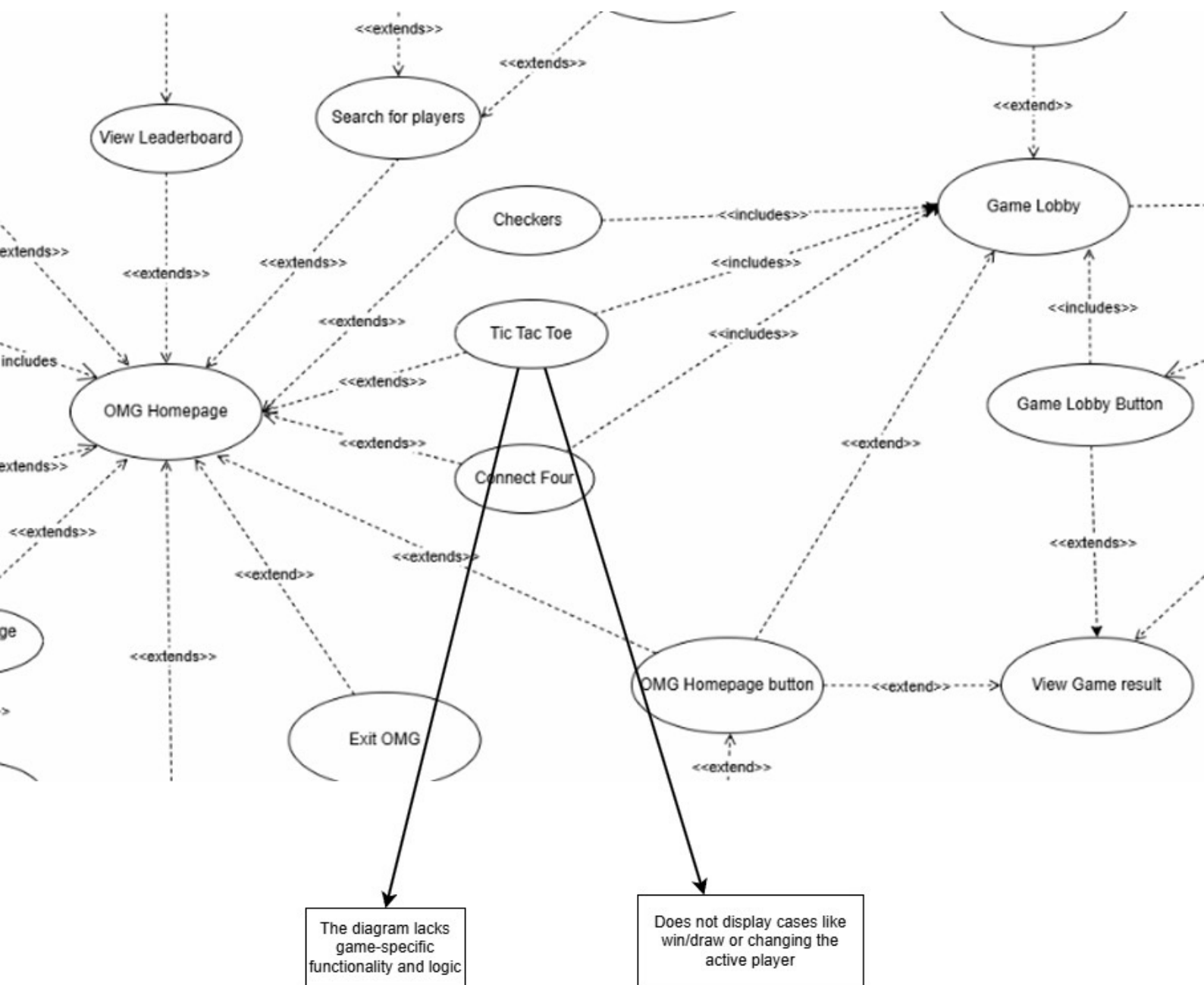
Channels to secondary actors: N/A

Open Issues:

1. Should there be a confirmation message after forfeiting to remind the user their match was recorded as a loss?
2. Should the system differentiate between a user intentionally forfeiting and disconnecting due to network issues?
3. If a player forfeits multiple times, should they be temporarily blocked from matchmaking?
4. If the opponent has already disconnected, does the player still receive a forfeit loss if they exit?

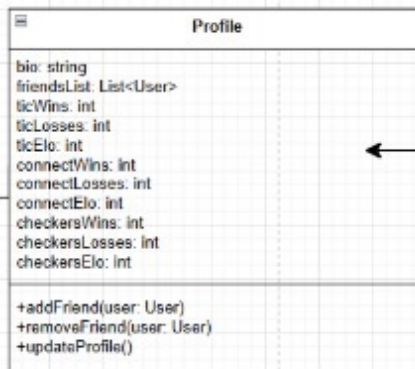
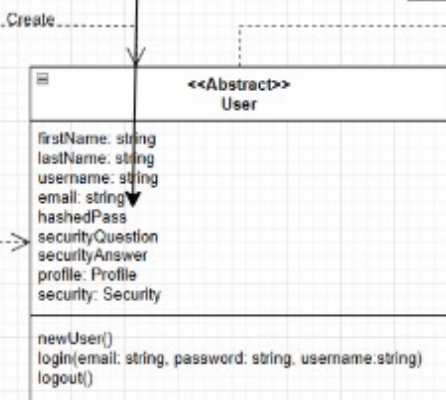
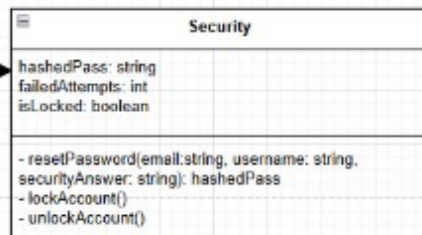
Should frequent forfeits result in matchmaking penalties, such as cooldowns or reduced ranking gains in future matches?





-hashedPass appears in both classes. They are connected with dependency, so there probably is not a need for it to appear twice

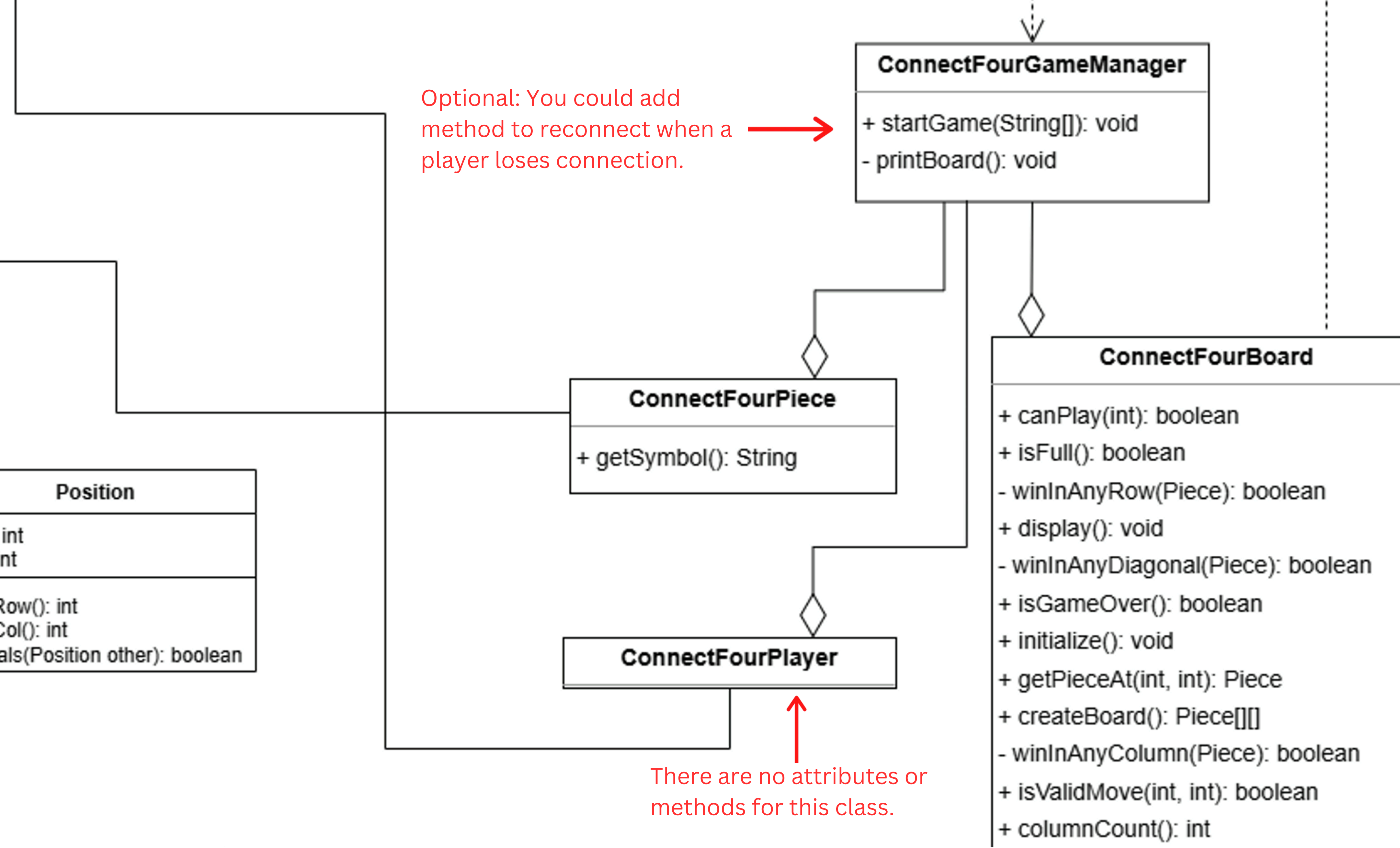
- Should not Security depend on User instead of the other way?



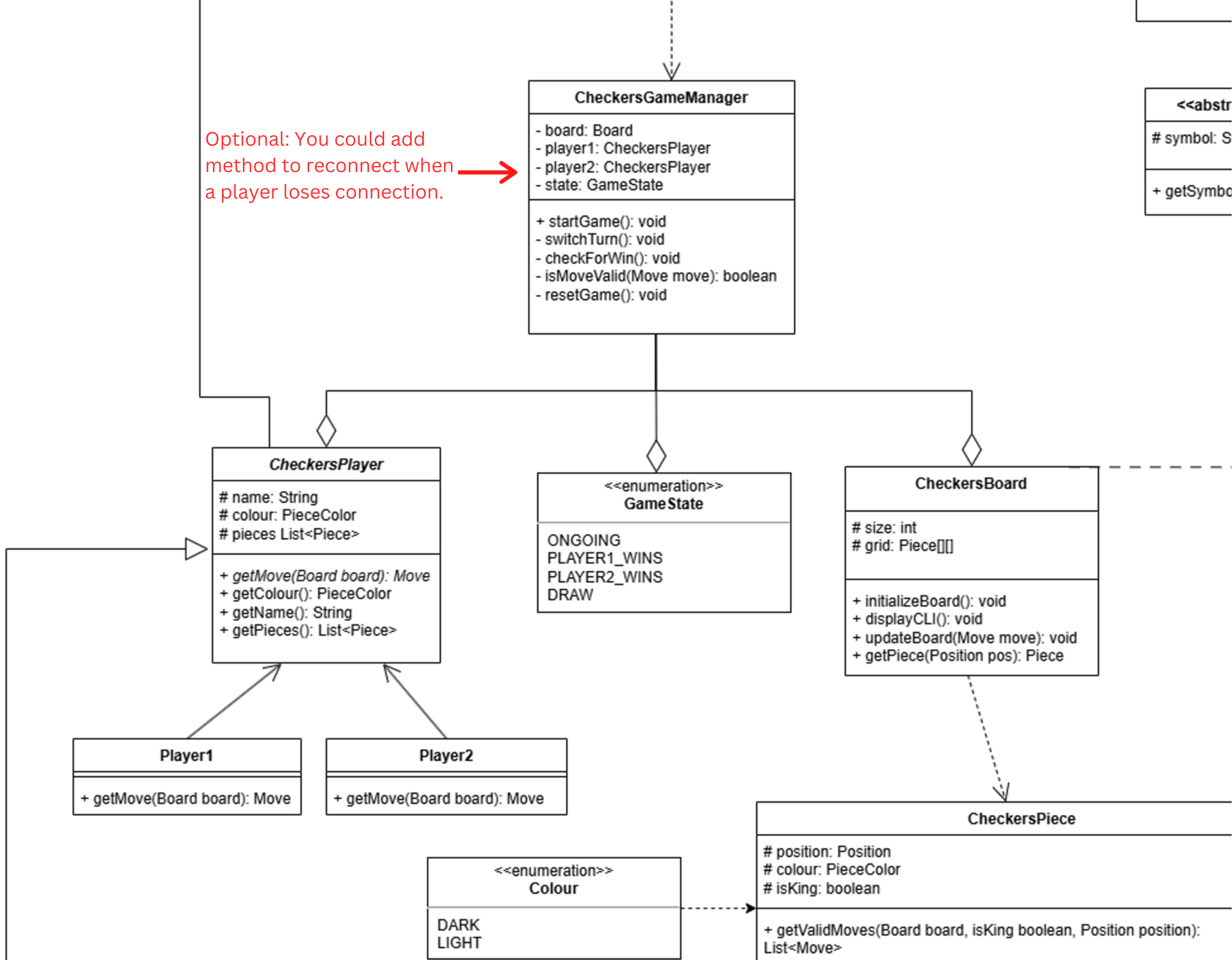
- Able to add another User as a friend, but does not provide any special features between Users that are friends. Recommend having a Friend class that extends User and have methods like message or whatever that only "Friends" can do together.

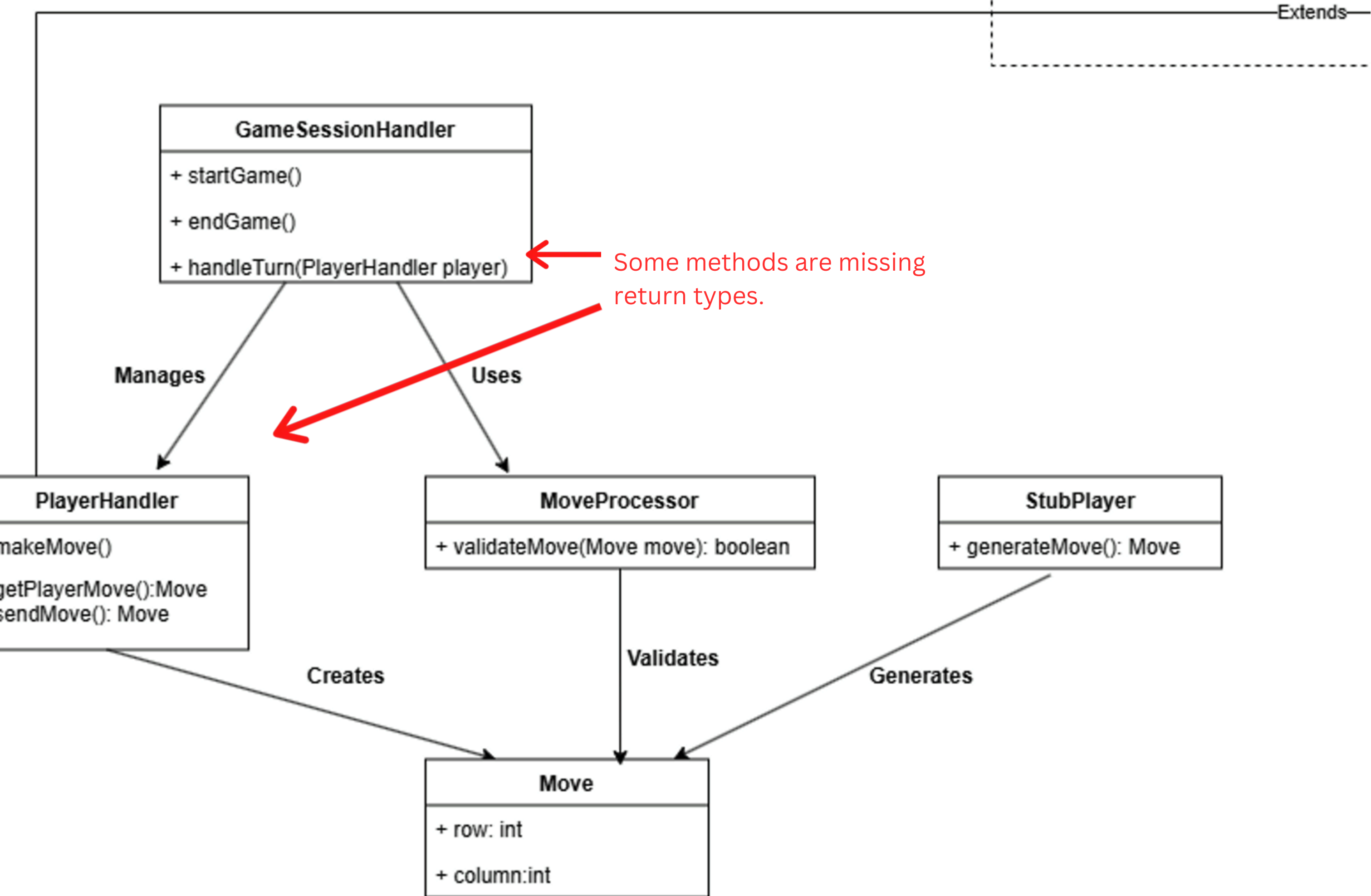
- Along with the wins and losses in each game, showing a calculated win rate in the profile will benefit user experience. (much easier/faster way to determine someone's skill level quickly alongside ELO)

- Displaying on/offline status and "last online" status information will help users have a smoother community interaction by providing the information on how active a certain user is on the platform.



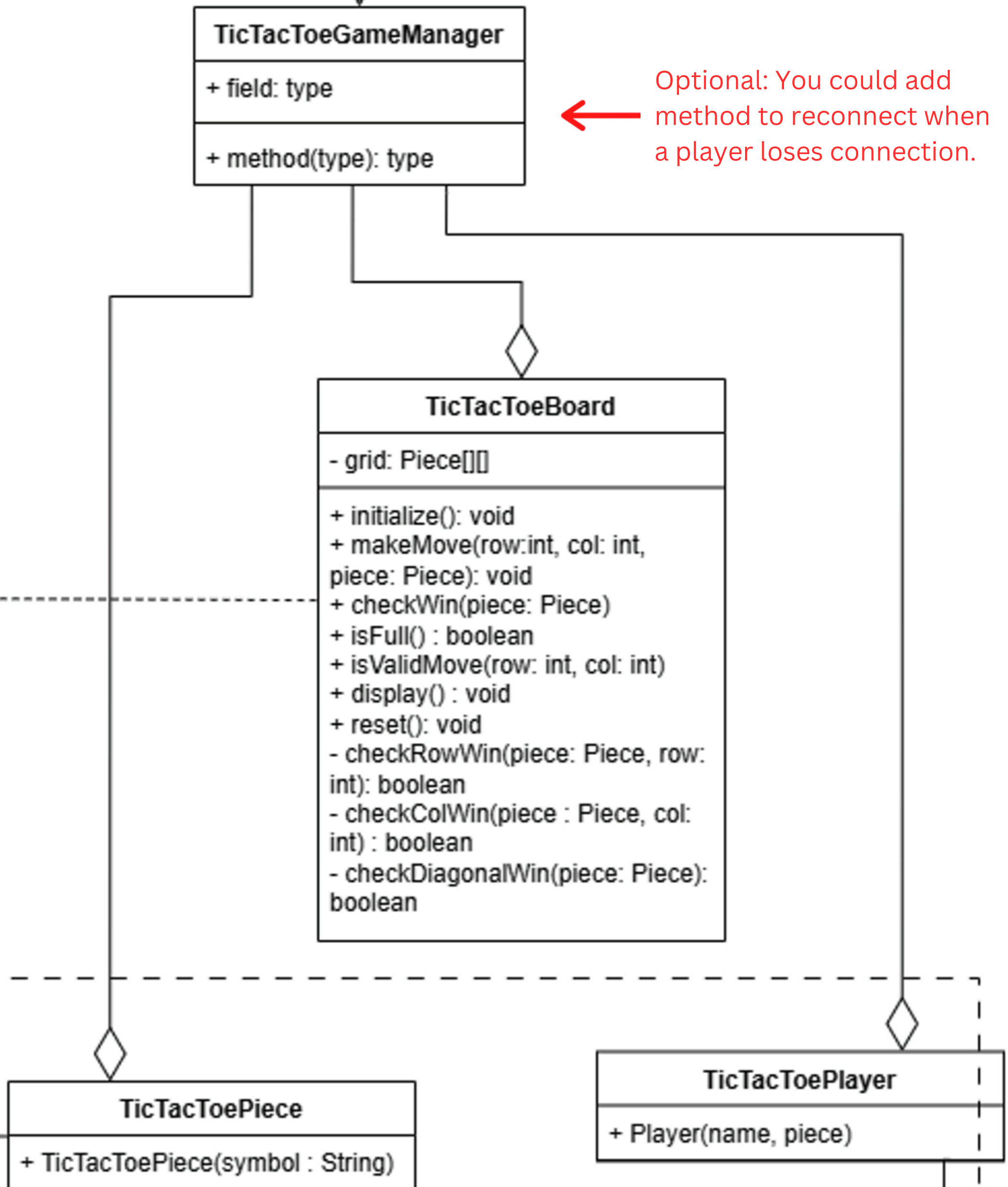
Optional: You could add method to reconnect when a player loses connection. →

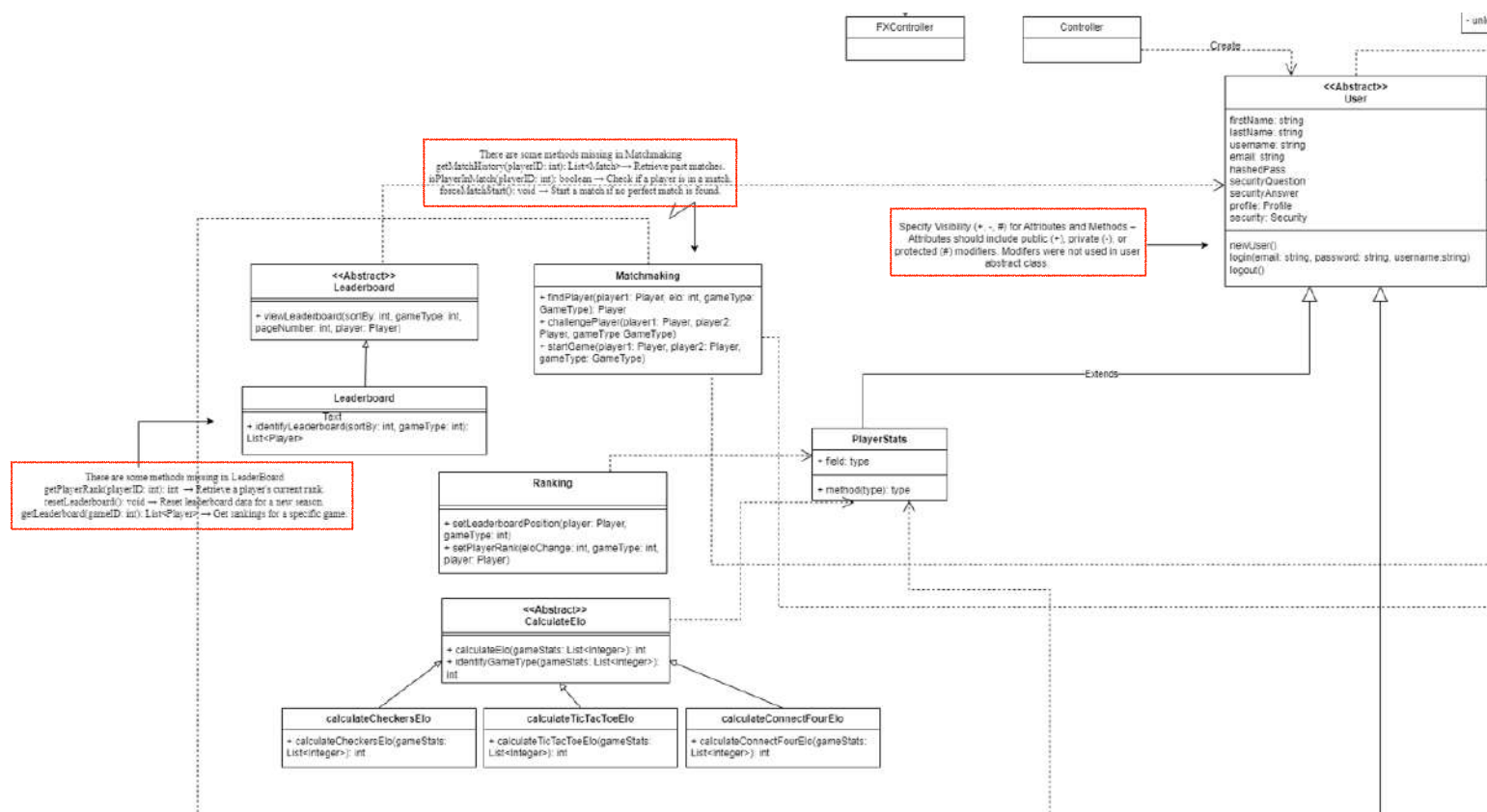


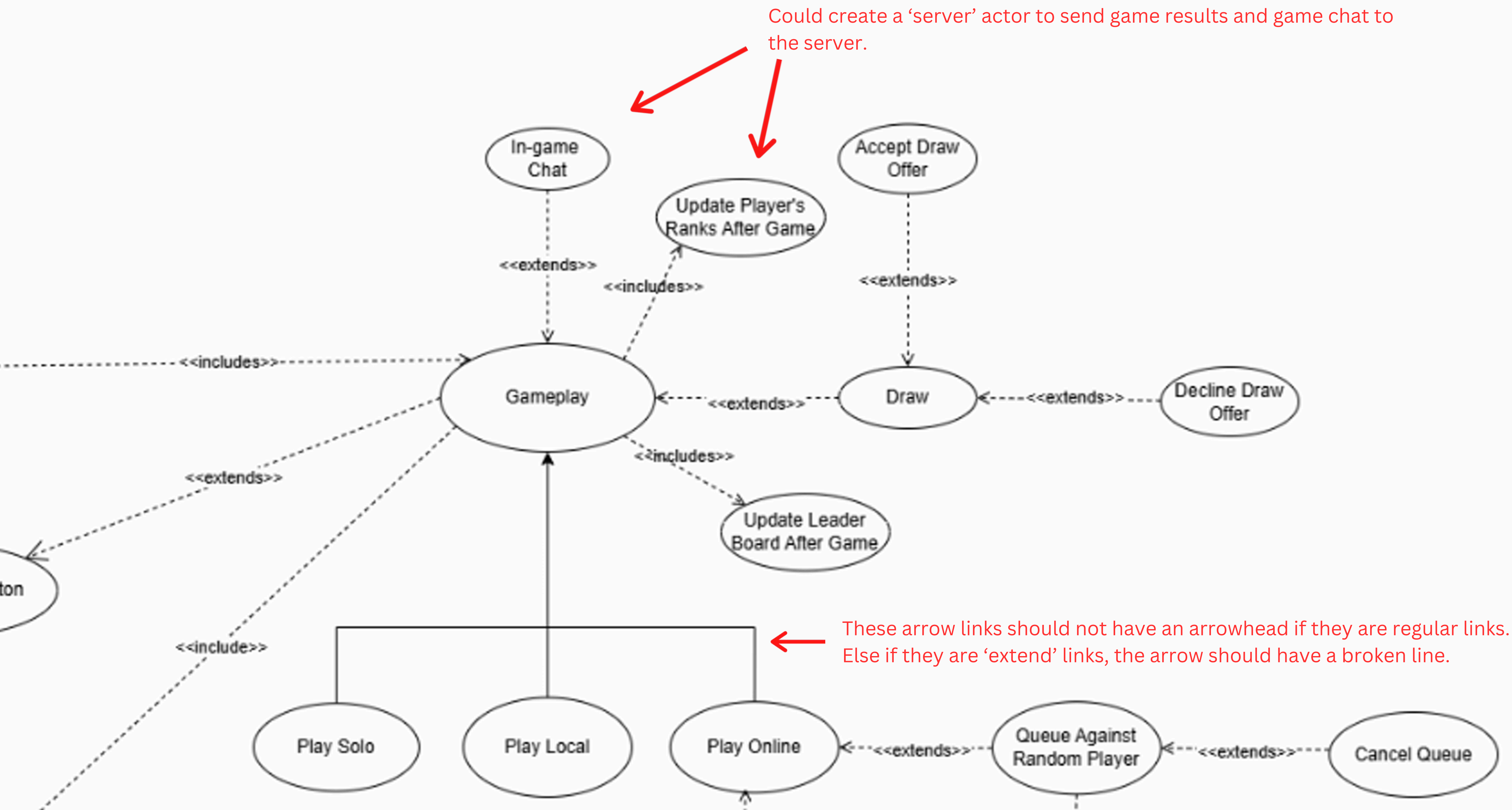


You could make separate diagrams for different components of the project to improve readability of the diagram.

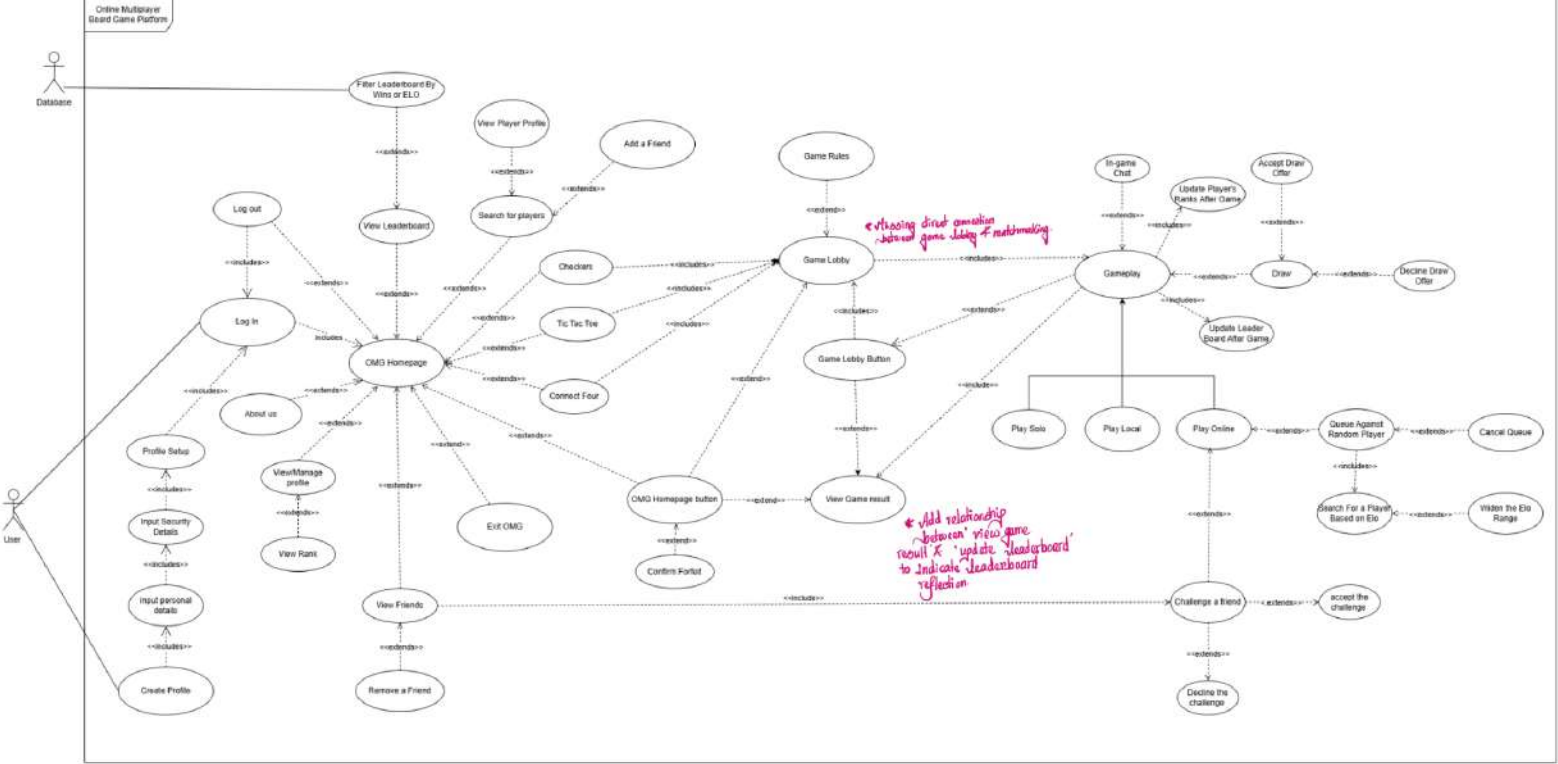
Optional: You could add method to reconnect when a player loses connection.

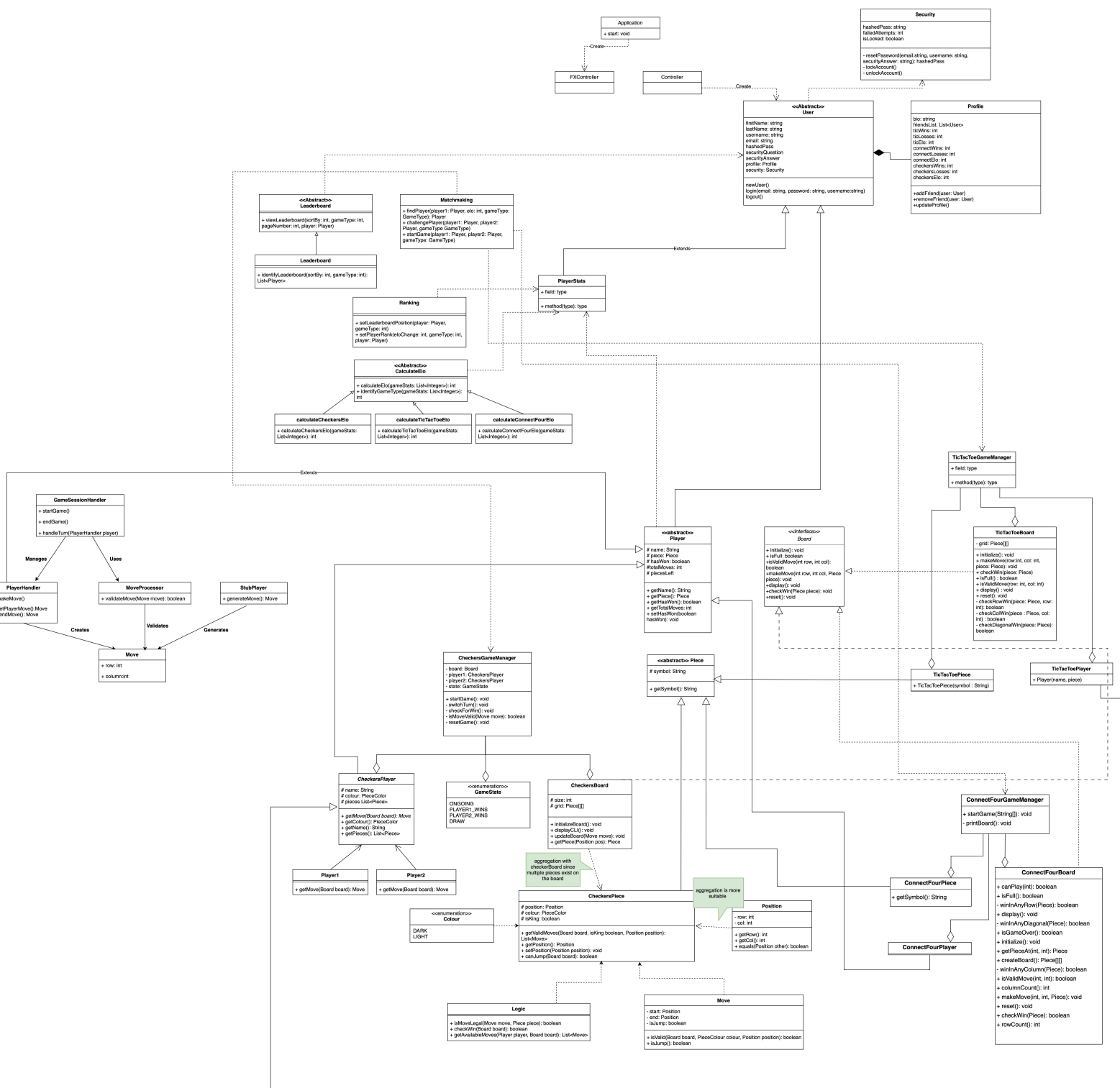






Use Case Diagram





23. Use case: Find opponent based on ELO

Iteration: 1

Primary Actor: Matchmaking System

Goal: Match a player with an opponent of similar skill level using ELO rating.

Preconditions:

- A player has queued for a game.
- At least one other player is also queued in the same game type.

Trigger: A player enters the matchmaking queue.

Scenario:

1. The system checks the queue for an opponent with X ELO range.
2. If an opponent is found within the range, the match is created.
3. The system places both players in the game lobby.
4. if no opponent is found, the system waits for Y seconds before deciding whether to expand the search range.

Exceptions:

1. Matchmaking server is down.
2. Player cancels the queue before a match is found.

Priority: High

When available: Sprint 1

Frequency of use: Frequently (every time a players queues)

Channel to actor: System process

• The system checks for an opponent within the ELO Range, but it doesn't specify whether the range dynamically adjusts based on wait time!

↳ If a match is not found, should the search expand over time?

Secondary actors: N/A

Channels to secondary actors: N/A

Open issues:

1. Should players be notified when the system expands the range?

24. Use case: Expand Search Range

Iteration: 1

Primary Actor: Matchmaking System

Goal: Ensure players find matches by gradually expanding ELO requirements

Preconditions:

- A player has been in the queue for longer than Y seconds without finding a match
- No opponent was found within the initial ELO range.

Trigger: "Find Opponent based on ELO" fails to find a match.

Scenario:

1. The system checks if the player has been waiting for more than certain seconds.
2. The system expands the allowed ELO range(e.g., +50 -> +100 ->200).
3. The system searches again for an opponent within the new range.
4. If a match is found, both players are placed in the game lobby.
5. If still no match is found, the system continues expanding the range in intervals.

Exceptions:

1. If all available opponents disconnect, the system stops searching.

Priority: High

When available: Within 2 sprints

- Instead of just asking if players should be notified, consider adding a UI indicator that warns players when system is broadening search range.

29

Frequency of use: Occasionally (only when matchmaking takes too long)

Channel to actor: System Process

Secondary actors: N/A

Channels to secondary actors: N/A

Open issues:

1. Should the player be notified when the search range expands?

Channel to actor: Game user interface

Secondary actors: N/A

Channels to secondary actors: N/A

Open issues:

1. How detailed should the statistics collected be?
2. How many match histories should be stored at a time?

39. Use case: Filter Leaderboard by Wins or ELO

Iteration: 1

Primary Actor: Player

Goal: Allow the player to filter the leaderboard depending on ELO or Wins

Preconditions:

1. Player is logged in
2. Leaderboard database is operational

Trigger: Player presses on drop down menu and selects whether to filter to leaderboard by wins or elo

Scenario:

1. Player navigates to the leaderboard page
2. Player clicks on the drop down menu
3. Player selects one of the two options being: "Filter by Wins" or "Filter by ELO"
4. Depending on the request the system fetches the appropriate data from the database

46

5. The system displays the leaderboard based on the player's preference
6. Player view's the filtered leaderboard

Exceptions:

1. If the database is unresponsive or takes too long to respond and error message will be displayed

Priority: Medium

→ *this feature is classified as medium priority. But filtering leaderboard data needs to be high priority as it might be critical if players want quick access to rankings*

When available: 1st or 2nd Iteration

Frequency of use: Occasionally, when players want to track where they stand against the competition

Channel to actor: Game user interface

Secondary actors: N/A

Channels to secondary actors: N/A

Open issues:

1. How should we approach the process of fetching the data in order to improve speed and performance?